



Bitácora de Prompts

-Inicio descripción de informe ejecutivo:

Prompt 1.

Actúa como consultor de sistemas de información y experiencia de usuario. Usando las fuentes cargadas (el Caso 06 "Rutas Enredadas" y el material disponible), redacta un INFORME EJECUTIVO para el diseño de la aplicación "RutaSmart".

CONTEXTO DEL PROBLEMA (úsalo como eje del informe):

RutaSmart resuelve el caso de Matías, un repartidor independiente de 22 años que entrega almuerzos caseros en bicicleta a oficinas de Santiago de Chile. Hoy gestiona entre 30 y 50 pedidos diarios de forma manual, con papeles, lo que le genera estos dolores (pain points): pierde tiempo planificando, se equivoca en el orden de entrega, no tiene visibilidad de qué zonas concentran la demanda ni a qué horas, y sus clientes (oficinistas) no pueden seguir su pedido ni saber cuándo llega. La app debe recibir pedidos, agruparlos por zona, calcular una ruta optimizada y ofrecer un panel de analíticas, además de la vista del cliente para pedir y hacer seguimiento.

ESTRUCTURA DEL INFORME (tres pilares):

1. NEURO-MARKETING Y PSICOLOGÍA DEL CONSUMO GASTRONÓMICO

- Sintetiza investigaciones sobre Menu Engineering (Ingeniería de Menús) y el efecto del diseño visual en la rentabilidad.
- Identifica factores de Psicología de la Elección (paradoja de la elección, efecto anclaje) que influyen en el ticket promedio.
- Extrae evidencia científica sobre cómo la interactividad y el material visual afectan la percepción de frescura y calidad del producto.
- Analiza estrategias de fidelización basadas en la experiencia de usuario (UX) durante el flujo de la visita y el pedido.

2. BENCHMARKING DE APLICACIONES LÍDERES Y PATRONES DE ÉXITO

- Identifica las funcionalidades "core" de las apps líderes (personalización de platos, pedidos en tiempo real, integración de pagos).
- Analiza los patrones de interfaz (UI) que facilitan la navegación rápida y reducen



la fricción en el pedido.

- Determina qué enfoques de gamificación o recompensas logran mayor retención.

3. "PAIN POINTS" Y EXPECTATIVAS DEL USUARIO FINAL

- Basado en reviews y foros, identifica los errores críticos de UX que causan abandono (lentitud, registro complejo, falta de fotos reales).
- Enumera funcionalidades que los usuarios consideran indispensables pero rara vez encuentran (filtros de alérgenos precisos, historial de pedidos, división de cuentas).
- Analiza la fricción entre el usuario digital y el servicio humano: qué esperan los clientes de la interacción con el repartidor cuando usan una app.

REQUISITOS DE SALIDA:

- Al final de cada pilar, incluye una sección "INSIGHTS ACCIONABLES" que conecte la teoría con funcionalidades concretas de RutaSmart (por ejemplo: seguimiento en tiempo real, dashboard con rutas optimizadas, confirmaciones visuales de entrega).
- Conecta explícitamente cada insight con los dolores de Matías y de sus clientes descritos en el contexto.
- Cierra el informe con un RESUMEN EJECUTIVO breve y una lista priorizada de las funcionalidades recomendadas para el prototipo.
- Tono profesional, claro y basado en las fuentes. Cita o referencia los documentos cargados cuando corresponda.
- Redáctalo en español.



-Prompts de diseño y creación de Stich.IA

Prompt.2

Diseña un prototipo de aplicación móvil llamada "RutaSmart", para un negocio de reparto de almuerzos caseros en bicicleta. El usuario es Matías, que reparte, y sus clientes son oficinistas que piden almuerzo.

Crea estas 3 pantallas principales con navegación entre ellas mediante una barra inferior de 3 íconos:

1. PANEL (Dashboard): tarjetas con indicadores del día (pedidos totales, pendientes, entregados, ingresos) y tres gráficos: pedidos por zona, demanda por hora del día, y pedidos por día de la semana.
2. RUTA DE HOY: una lista vertical numerada de entregas ordenadas por cercanía, agrupadas por zona con un color por zona. Cada entrega muestra cliente, dirección y un botón para marcar "Entregado". Arriba, una barra de progreso de la ruta.
3. HACER PEDIDO: un menú de productos con foto, nombre y precio; el cliente agrega productos a un carrito, ve el total y confirma el pedido.

Estilo visual: cálido y apetitoso, paleta tierra (tomate, verde, mostaza) sobre fondo crema, tipografía moderna y legible. Diseño limpio, tipo app de delivery.



-Prompt Diseño de DASHBOARD de la aplicación

Prompt.3

Crea una aplicación web responsive llamada "RutaSmart", para un negocio de reparto de almuerzos caseros en bicicleta en Santiago de Chile. El usuario principal es Matías, el repartidor, y también la usan sus clientes (oficinistas). La app optimiza las rutas de entrega y muestra analíticas del negocio. Toda la interfaz está en español.

ESTILO VISUAL:

- Paleta: azul profundo como color primario, turquesa/cian como acento, fondo blanco y gris muy claro, texto azul marino oscuro. Tarjetas con esquinas redondeadas y sombras suaves. Diseño limpio, moderno y profesional, tipo app móvil.
- Tipografía moderna sin serifa, legible. Íconos de línea simples.
- Barra de navegación inferior fija con 5 secciones: Inicio, Ruta, Mapa, Analíticas y Perfil.

PANTALLAS:

1. INICIO (Dashboard): Saludo "Buenas noches, Matías". Cuatro tarjetas con métricas del día: Pedidos hoy (42), Completados (28), Pendientes (14) y Tiempo de jornada (6h 15m). Una tarjeta destacada de "Resumen Inteligente" que indica la zona y hora pico del día (ej: "Centro Histórico, 11:00 AM"). Debajo, tres accesos rápidos en tarjetas: "Hoja de Ruta Diaria" (botón Ver Ruta), "Mapa de Calor" (botón Ver Mapa) y "Análisis de Demanda" (botón Ver Análisis).
2. RUTA (Hoja de Ruta): Encabezado con tres datos: total de pedidos (42), tiempo estimado de jornada (6h 15m) y distancia total (15.3 km). Un mapa con la ruta trazada y un botón "Optimizar". Debajo, una lista vertical numerada de las paradas de entrega ordenadas por cercanía (ej: Av. Santa Fe 1230, Buines 2840, Carrera 950), cada una con su dirección, hora estimada y estado (Entregado / En ruta / Pendiente). Un enlace "Ver 39 paradas más" y un botón destacado "Iniciar Ruta".
3. MAPA (Mapa de Calor): Título "Demanda por Zonas" con filtros de Hoy / Semana / Mes. Un mapa de calor que muestra las zonas con más pedidos en tonos más intensos.



Debajo, un "Ranking de zonas" con porcentajes: Centro (38%), Providencia (30%), Las Condes, etc. Una nota de tendencia (ej: "Crecimiento zona norte +10%") y un botón "Ir Ahora".

4. ANALÍTICAS (Análisis de Demanda): Título "Análisis de Rendimiento". Métricas: volumen total semanal (1250 pedidos), rendimiento (180 pedidos/día) y hora pico (11:00 AM). Un gráfico de barras "Pedidos por día" (lunes a domingo). Una tarjeta con la conclusión clave: "La franja de 12:00 a 14:00 presenta la mayor demanda" y una etiqueta "Optimización Alta". Un gráfico de línea "Flujo Horario (08:00-20:00)" mostrando la demanda por hora.
5. PERFIL (Perfil del Usuario): Foto y nombre "Matías Aravena", rol "Repartidor". Información personal: correo, medio de transporte (Bicicleta) y comuna (Providencia). Una tarjeta de "Estado: Activo" con horario. Métricas del repartidor: total de pedidos (1842), kilómetros recorridos (3125 km), tiempo promedio por entrega (18 min), calificación (4.8/5), cumplimiento (98%) y días activos (547). Una sección de Logros con insignias. Y un menú de Configuración: Editar Perfil, Contraseña, Notificaciones, Privacidad, Ayuda y Cerrar Sesión.

LÓGICA Y DATOS DE EJEMPLO:

- Genera datos de prueba realistas de Santiago de Chile: 6 zonas (Centro, Providencia, Las Condes, Ñuñoa, Santiago Poniente, Vitacura), un menú de almuerzos caseros chilenos con precios en pesos chilenos, y unos 40 pedidos del día con horas concentradas entre las 11:00 y las 14:00.
- La ruta debe ordenar las entregas agrupándolas por zona y por cercanía geográfica (la entrega más cercana primero), partiendo desde la cocina de Matías.
- El repartidor puede marcar cada pedido como "Entregado" y ver el avance de la ruta.
- Las analíticas (pedidos por zona, demanda por hora, pedidos por día) deben calcularse a partir de esos datos de pedidos.

REQUISITO TÉCNICO:

- Aplicación web responsive que se vea bien en celular. Datos de ejemplo incluidos para que funcione de inmediato sin configuración.



-Prompt Diseño Visual

Prompt.4

Rediseña completamente la interfaz visual de RutaSmart con una estética moderna, minimalista y profesional, inspirada en apps SaaS de logística y análisis de datos.

PALETA DE COLORES:

Fondo principal: #FFFFFF	Fondo secundario: #F8FAFC
Títulos: #2563EB	Texto principal: #334155
Texto secundario: #64748B	Tarjetas y widgets: #BAE6FD
Botones principales: #3B82F6	Hover/destacados: #1D4ED8
Indicadores y mapas: #67E8F9	Bordes: #E2E8F0

INSTRUCCIONES DE DISEÑO:

1. Cambiar todos los fondos beige y crema por blanco (#FFFFFF).
2. Títulos en azul #2563EB, tipografía moderna, peso semibold/bold.
3. Tarjetas y paneles de estadísticas en celeste pastel #BAE6FD.
4. Mantener espacio en blanco para una percepción premium.
5. Bordes redondeados de 16px en tarjetas, botones y componentes.
6. Sombras suaves: box-shadow: 0 4px 12px rgba(37,99,235,0.08).
7. Botones principales: fondo #3B82F6, texto blanco, hover #1D4ED8.
8. Iconos activos en azul #2563EB o turquesa #67E8F9.
9. El mapa de calor mantiene su intensidad pero con controles blancos y sombras suaves.
10. Barra inferior: fondo blanco, icono activo #2563EB, fondo del ítem activo #BAE6FD, iconos inactivos #64748B.
11. Apariencia limpia y tecnológica (Google Maps, Notion, Stripe, Linear).
12. Priorizar accesibilidad, contraste y elegancia visual.

OBJETIVO FINAL: una interfaz fresca, minimalista y profesional, dominada por blancos, azules y celestes pastel, que transmita innovación, confianza y tecnología.



-Stack tecnológico

Prompt.5

Actúa como desarrollador full-stack. Crea la aplicación web completa "RutaSmart", para un negocio de reparto de almuerzos caseros en bicicleta en Santiago de Chile. El usuario principal es Matías (el repartidor) y también la usan sus clientes (oficinistas). La app debe recibir pedidos, agruparlos por zona, calcular una ruta de entrega optimizada y mostrar un panel de analíticas. Todo en español.

===== STACK TECNOLÓGICO =====

- Back-end: Node.js + Express.
- Base de datos: SQLite usando el módulo nativo de Node (node:sqlite), SIN librerías que requieran compilación nativa (no uses better-sqlite3).
- Front-end: HTML, CSS y JavaScript (sin frameworks pesados).
- El proyecto debe ejecutarse con un solo "npm install && npm start" y servir el front-end desde el mismo backend en `http://localhost:3000`.

===== BACK-END =====

Crea una API REST que devuelva y reciba datos en formato JSON, con estos endpoints:

- GET `/api/zonas` -> lista de zonas.
- GET `/api/productos` -> menú de productos.
- GET `/api/clientes` -> lista de clientes (id, nombre, zona).
- GET `/api/pedidos?fecha=...` -> pedidos de una fecha, con su detalle.
- POST `/api/pedidos` -> crea un pedido nuevo (de forma TRANSACCIONAL: si falla cualquier parte, no se guarda nada).
- PATCH `/api/pedidos/:id/estado` -> cambia el estado de un pedido (pendiente -> en_ruta -> entregado).
- GET `/api/ruta/hoy` -> la ruta optimizada del día.
- GET `/api/dashboard` -> indicadores: total de pedidos del día, pendientes, entregados, ingresos, pedidos por zona, demanda por hora y pedidos por día de los últimos 7 días.

Optimizador de rutas: agrupa los pedidos no entregados del día por zona y los ordena por cercanía geográfica (algoritmo del vecino más cercano usando distancia de



haversine), partiendo siempre desde la cocina de Matías. Devuelve las paradas en orden, la distancia total y el tiempo estimado.

Base de datos: crea las tablas zona, cliente, producto, repartidor, ruta, pedido y detalle_pedido (esta última resuelve la relación muchos-a-muchos entre pedido y producto, guardando cantidad y precio_unitario). Usa claves primarias y foráneas con integridad referencial. Carga datos de prueba al iniciar: 6 zonas (Centro, Providencia, Las Condes, Ñuñoa, Santiago Poniente, Vitacura), ~10 almuerzos caseros chilenos con precios en pesos, 12 clientes, 1 repartidor (Matías) y ~40 pedidos del día con las horas concentradas entre las 11:00 y las 14:00.

===== FRONT-END =====

Interfaz web responsive (que se vea bien en celular) que consuma la API anterior mediante fetch. Replica el diseño de mi prototipo: paleta azul profundo como color primario, turquesa/cian como acento, fondo blanco y gris claro, tarjetas con esquinas redondeadas y sombras suaves, diseño limpio tipo app de delivery. Barra de navegación inferior con las siguientes vistas:

1. PANEL (Dashboard): tarjetas con los indicadores del día (pedidos, completados, pendientes, tiempo) y tres gráficos: pedidos por zona, demanda por hora y pedidos por día. Estos gráficos responden las 3 preguntas del negocio.
2. RUTA DE HOY: lista vertical numerada de las entregas ordenadas por cercanía, agrupadas por zona (un color por zona), cada una con dirección, hora y estado. Un botón para marcar cada pedido como "Entregado", una barra de progreso de la ruta y un botón "Optimizar".
3. HACER PEDIDO: el cliente ve el menú con precios, agrega productos a un carrito, ve el total y confirma el pedido (que se envía al backend con POST /api/pedidos y aparece en la ruta).

Maneja los errores de red mostrando un mensaje claro al usuario.

===== ENTREGABLE =====



Entrega el proyecto organizado en carpetas: /backend (server, conexión a la base de datos y optimizador), /frontend (index.html, estilos y JavaScript) y /db (scripts SQL de estructura y datos). Incluye un README con el stack y las instrucciones para ejecutarlo.

-Códigos Vista Clientes

Prompts.6

Actúa como desarrollador full-stack. Amplía la aplicación "RutaSmart" (reparto de almuerzos caseros en bicicleta en Santiago de Chile) para implementar el FLUJO COMPLETO de un pedido, desde que el cliente lo crea hasta que aparece en la vista del repartidor para iniciar la entrega. Todo en español.

STACK (mantener el existente):

- Back-end: Node.js + Express.
- Base de datos: SQLite con el módulo nativo node:sqlite (sin librerías que compilen binarios nativos).
- Front-end: HTML, CSS y JavaScript, con la paleta azul profundo + turquesa sobre fondo oscuro/claro, tarjetas redondeadas, tipo app de delivery.
- Debe correr con "npm install && npm start".

===== FLUJO DEL PEDIDO (lo más importante) =====

El pedido pasa por estos estados, en este orden:

pendiente -> confirmado -> en_ruta -> entregado

- "pendiente": el cliente lo está armando (aún en el carrito, no enviado).
- "confirmado": el cliente lo confirmó; queda DISPONIBLE para el repartidor.
- "en_ruta": el repartidor lo aceptó e inició la entrega.
- "entregado": el repartidor marcó la entrega como completada.

===== BACK-END (API REST) =====

Crea/ajusta estos endpoints en JSON:

- GET /api/productos -> menú de almuerzos (id, nombre, descripción, precio, categoría, imagen).
- POST /api/pedidos -> crea un pedido con sus productos y cantidades.



- Nace en estado "confirmado". Debe ser
TRANSACCIONAL: si falla una parte, no se guarda
nada. Calcula y guarda el total.
- GET /api/pedidos/disponibles -> lista los pedidos en estado "confirmado" (los que el repartidor puede tomar), con cliente, zona, productos y total.
 - PATCH /api/pedidos/:id/aceptar -> el repartidor acepta el pedido: pasa a "en_ruta" y dispara el cálculo de la ruta.
 - PATCH /api/pedidos/:id/entregar -> marca el pedido como "entregado".
 - GET /api/ruta/hoj -> devuelve la ruta optimizada con los pedidos "en_ruta", ordenados por cercanía (vecino más cercano con distancia de haversine) desde la cocina de Matías, con distancia total y orden de paradas.

Base de datos: tablas zona, cliente, producto, repartidor, pedido y detalle_pedido (relación muchos-a-muchos entre pedido y producto, con cantidad y precio_unitario), con claves foráneas e integridad referencial. Incluye datos de prueba: 6 zonas de Santiago, los almuerzos caseros chilenos del menú (Pastel de Choclo, Cazuela de Vacuno, Porotos con Riendas, Charquicán, Humitas, Ensalada César, etc.) con precios en pesos, varios clientes y el repartidor Matías.

===== FRONT-END: VISTA DEL CLIENTE =====

1. MENÚ: lista de almuerzos con foto, nombre, descripción y precio. Cada producto con un BOTÓN "Agregar al carrito" (y controles + / - para la cantidad).
2. CARRITO: muestra los productos elegidos, permite ajustar cantidades, muestra el TOTAL, tiene un campo de dirección/zona y un BOTÓN "Confirmar pedido" que llama a POST /api/pedidos.
3. CONFIRMACIÓN: mensaje "¡Pedido confirmado!" con el número de pedido y un BOTÓN "Seguir mi pedido" que muestra el estado actual.

===== FRONT-END: VISTA DEL REPARTIDOR =====

4. PEDIDOS DISPONIBLES: lista en tiempo real los pedidos en estado "confirmado" (consumiendo GET /api/pedidos/disponibles). Cada tarjeta muestra cliente, zona,



productos y total, con un BOTÓN "Aceptar e iniciar entrega" que llama a
PATCH /api/pedidos/:id/aceptar.

5. RUTA DE HOY: al aceptar, el pedido entra a la ruta optimizada. Muestra las paradas en orden por cercanía, con un BOTÓN "Marcar como entregado" en cada una (PATCH /api/pedidos/:id/entregar) y una barra de progreso de la ruta.

===== CONEXIÓN =====

Lo esencial: cuando el cliente confirma su pedido en la vista del cliente, ese pedido debe aparecer automáticamente en "Pedidos disponibles" de la vista del repartidor (se comunican a través de la misma base de datos vía la API). Al aceptarlo, se recalcula la ruta. Permite alternar entre la vista del cliente y la del repartidor con un botón "Cambiar perspectiva".

Entrega el proyecto en carpetas /backend, /frontend y /db, con un README de ejecución.

-Dashboar de la vista del repartidor

Prompt.6

Actúa como desarrollador full-stack. Crea la aplicación "RutaSmart" desde la PERSPECTIVA DEL REPARTIDOR (Matías), que reparte almuerzos caseros en bicicleta en Santiago de Chile. La app resuelve sus dolores: pierde tiempo ordenando pedidos a mano, se equivoca en el orden de entrega y no tiene visibilidad de qué zonas u horas concentran la demanda. Todo en español.

STACK:

- Back-end: Node.js + Express.
- Base de datos: SQLite (usa el esquema SQL que entrego más abajo).
- Front-end: HTML, CSS y JavaScript responsive. Paleta azul profundo + turquesa, fondo oscuro/claro, tarjetas redondeadas, barra de navegación inferior. Tipo app de delivery moderna.
- Debe ejecutarse de inmediato con datos de ejemplo.

===== LÓGICA DEL BACK-END (REPARTIDOR) =====



Implementa el ciclo de trabajo de Matías con estos endpoints REST en JSON:

- GET /api/pedidos/disponibles -> pedidos en estado "confirmado" listos para tomar.
- PATCH /api/pedidos/:id/aceptar -> TOMA DEL PEDIDO: pasa el pedido a "en_ruta" y dispara el recálculo de la ruta del día.
- PATCH /api/pedidos/:id/entregar -> CONFIRMA LA ENTREGA: pasa el pedido a "entregado" y actualiza el progreso de la ruta.
- GET /api/ruta/hoy -> CÁLCULO DE RUTA: ordena los pedidos "en_ruta" agrupándolos por zona y por cercanía geográfica (algoritmo del vecino más cercano con distancia de haversine), partiendo desde la cocina de Matías. Devuelve las paradas en orden, distancia total y tiempo estimado.
- GET /api/dashboard -> indicadores del repartidor (ver abajo).

===== VISUALIZACIONES DEL REPARTIDOR (5 vistas) =====

1. INICIO (Dashboard): tarjetas con los indicadores del día (pedidos totales, completados, pendientes, tiempo de jornada, ingresos) y un "resumen inteligente" con la zona y hora pico. Accesos rápidos a Ruta, Mapa y Análisis.
2. HOJA DE RUTA: lista numerada de entregas ordenadas por cercanía, agrupadas por zona (un color por zona), cada una con dirección, hora y estado. Botón "Marcar como entregado" por parada, barra de progreso de la ruta y botón "Optimizar".
3. PEDIDOS DISPONIBLES: pedidos "confirmados" que aún nadie tomó, cada uno con un botón "Aceptar e iniciar entrega".
4. MAPA DE CALOR: demanda por zonas con un ranking (Centro, Providencia, Las Condes...) en porcentajes. Resuelve "¿qué zonas generan más pedidos?".
5. ANÁLISIS DE DEMANDA: gráfico de pedidos por día (últimos 7 días) y de demanda por hora, con la franja horaria pico destacada. Resuelve "¿a qué hora hay más demanda?".
6. PERFIL: datos de Matías (correo, transporte: bicicleta, comuna), métricas acumuladas (pedidos totales, km recorridos, tiempo promedio, calificación) y configuración.

===== ESQUEMA SQL (úsalo tal cual) =====

-- DDL: estructura de la base de datos

CREATE TABLE zona (



```
    id INTEGER PRIMARY KEY,
    nombre TEXT NOT NULL UNIQUE,
    color TEXT NOT NULL,
    centro_lat REAL NOT NULL,
    centro_lng REAL NOT NULL
);

CREATE TABLE cliente (
    id INTEGER PRIMARY KEY,
    nombre TEXT NOT NULL,
    direccion TEXT NOT NULL,
    zona_id INTEGER NOT NULL REFERENCES zona(id),
    lat REAL NOT NULL,
    lng REAL NOT NULL
);

CREATE TABLE producto (
    id INTEGER PRIMARY KEY,
    nombre TEXT NOT NULL,
    descripcion TEXT,
    precio INTEGER NOT NULL,
    categoria TEXT NOT NULL
);

CREATE TABLE repartidor (
    id INTEGER PRIMARY KEY,
    nombre TEXT NOT NULL,
    origen_lat REAL NOT NULL,
    origen_lng REAL NOT NULL
);

CREATE TABLE pedido (
    id INTEGER PRIMARY KEY,
    cliente_id INTEGER NOT NULL REFERENCES cliente(id),
    zona_id INTEGER NOT NULL REFERENCES zona(id),
    fecha TEXT NOT NULL,
    hora INTEGER NOT NULL,
    estado TEXT NOT NULL DEFAULT 'confirmado', -- confirmado | en_ruta | entregado
```



```
total INTEGER NOT NULL DEFAULT 0,
orden_ruta INTEGER
);
CREATE TABLE detalle_pedido (
    id INTEGER PRIMARY KEY,
    pedido_id INTEGER NOT NULL REFERENCES pedido(id),
    producto_id INTEGER NOT NULL REFERENCES producto(id),
    cantidad INTEGER NOT NULL,
    precio_unitario INTEGER NOT NULL
);

-- Vistas para las analíticas del dashboard
CREATE VIEW v_pedidos_por_zona AS
    SELECT z.nombre, z.color, COUNT(p.id) AS total
    FROM pedido p JOIN zona z ON z.id = p.zona_id
    GROUP BY z.id ORDER BY total DESC;
CREATE VIEW v_demanda_por_hora AS
    SELECT hora, COUNT(*) AS total
    FROM pedido GROUP BY hora ORDER BY hora;

-- DML: datos de prueba iniciales
INSERT INTO zona (id,nombre,color,centro_lat,centro_lng) VALUES
    (1,'Centro','#2E5BFF',-33.445,-70.650),
    (2,'Providencia','#17C3B2',-33.427,-70.610),
    (3,'Las Condes','#3A86FF',-33.408,-70.570),
    (4,'Ñuñoa','#4895EF',-33.456,-70.600),
    (5,'Santiago Poniente','#5390D9',-33.450,-70.690),
    (6,'Vitacura','#48BFE3',-33.390,-70.560);
INSERT INTO repartidor (id,nombre,origen_lat,origen_lng) VALUES
    (1,'Matías Aravena',-33.4489,-70.6693);
INSERT INTO producto (id,nombre,descripcion,precio,categoria) VALUES
    (1,'Pastel de Choclo Casero','Tradicional pastel chileno horneado en greda',6800,'Platos Caseros'),
```



```
(2,'Cazuela de Vacuno','Caldo casero concentrado de vacuno con choclo y  
zapallo',6200,'Platos Caseros'),  
  
(3,'Porotos con Riendas','Porotos guisados con tallarines y longaniza',5900,'Platos  
Caseros'),  
  
(4,'Charquicán de Verduras','Guiso de papas machacadas con huevo  
frito',5200,'Vegetarianos'),  
  
(5,'Humitas de la Casa','Choclo tierno molido envuelto en hojas (2  
unidades)',4900,'Vegetarianos'),  
  
(6,'Ensalada César','Lechugas frescas con aderezo César y garbanzos  
tostados',4600,'Vegetarianos'),  
  
(7,'Bebida 350ml','Bebida en lata fría',1500,'Bebidas');
```

```
-- Genera además ~40 pedidos de ejemplo del día de hoy, con sus detalles, repartidos  
-- en las 6 zonas y con las horas concentradas entre las 11:00 y las 14:00. Algunos  
-- en estado "confirmado" (para Pedidos Disponibles), algunos "en_ruta" y algunos  
-- "entregado", para que el dashboard, la ruta y las analíticas tengan datos.
```

===== ENTREGABLE =====

Organiza el proyecto en /backend, /frontend y /db, e incluye un README con el stack y las instrucciones de ejecución. Asegúrate de que el dashboard, la hoja de ruta y las analíticas se calculen a partir de los datos de la base.

-prompt Unión de ambas visualizaciones

Prompt.7

Actúa como desarrollador full-stack. Crea la aplicación web completa "RutaSmart", un sistema de reparto de almuerzos caseros en bicicleta en Santiago de Chile, con DOS PERSPECTIVAS conectadas: la del CLIENTE (oficinista que pide su almuerzo) y la del REPARTIDOR (Matías, que entrega). Ambas comparten el mismo back-end y la misma base de datos. Todo en español.

STACK:

- Back-end: Node.js + Express.
- Base de datos: SQLite (usa el esquema SQL que entrego más abajo, tal cual).
- Front-end: HTML, CSS y JavaScript responsive. Paleta azul profundo + turquesa sobre fondo oscuro/claro, tarjetas redondeadas, tipo app de delivery moderna.



- Debe ejecutarse de inmediato con los datos de ejemplo.
- Incluye un botón "Cambiar perspectiva" para alternar entre la vista del Cliente y la vista del Repartidor.

===== CICLO DEL PEDIDO (clave del sistema) =====

Estados del pedido, en orden:

confirmado -> en_ruta -> entregado

- El CLIENTE crea el pedido: nace "confirmado" y queda DISPONIBLE para el repartidor.
- El REPARTIDOR lo acepta: pasa a "en_ruta" y se recalcula la ruta.
- El REPARTIDOR lo entrega: pasa a "entregado" y avanza la barra de progreso.

Lo esencial: cuando el cliente confirma su pedido, este debe aparecer automáticamente en "Pedidos Disponibles" de la vista del repartidor. Las dos vistas se comunican a través de la misma base de datos vía la API.

===== BACK-END (API REST en JSON) =====

- GET /api/productos -> menú de almuerzos (id, nombre, descripción, precio, categoría).
- POST /api/pedidos -> el CLIENTE crea un pedido con sus productos y cantidades. Nace "confirmado". TRANSACCIONAL: si falla una parte, no se guarda nada. Calcula el total.
- GET /api/pedidos/:id -> estado y detalle de un pedido (para el seguimiento del cliente).
- GET /api/pedidos/disponibles -> pedidos "confirmados" listos para que el repartidor los tome (con cliente, zona, productos y total).
- PATCH /api/pedidos/:id/aceptar -> el REPARTIDOR toma el pedido: pasa a "en_ruta" y dispara el cálculo de la ruta.
- PATCH /api/pedidos/:id/entregar -> el REPARTIDOR confirma la entrega: pasa a "entregado".
- GET /api/ruta/hoy -> ruta optimizada con los pedidos "en_ruta", agrupados por zona y ordenados por cercanía (vecino más cercano con distancia de haversine) desde la cocina de Matías. Devuelve paradas en



orden, distancia total y tiempo estimado.

- GET /api/dashboard -> indicadores del repartidor: pedidos del día, pendientes, entregados, ingresos, pedidos por zona, demanda por hora y pedidos por día.

===== FRONT-END: VISTA DEL CLIENTE =====

Barra inferior con: Inicio, Menú, Mis Pedidos, Perfil.

1. INICIO: saludo "Hola, [nombre]", tarjeta del "Almuerzo del día" y botón "Pedir ahora". Si hay un pedido activo, muestra arriba su seguimiento.
2. MENÚ: almuerzos caseros chilenos por categoría (Platos Caseros, Vegetarianos, Bebidas), cada uno con descripción, precio y botón "Agregar al carrito" (con controles + / -).
3. CARRITO: productos elegidos, ajuste de cantidades, TOTAL, campo de dirección/zona y botón "Confirmar pedido" (llama a POST /api/pedidos).
4. CONFIRMACIÓN: "¡Pedido confirmado!", número de pedido y botón "Seguir mi pedido".
5. SEGUIMIENTO EN TIEMPO REAL: línea de progreso del pedido (Recibido -> En preparación -> En camino -> Entregado) y tiempo estimado de llegada.
6. MIS PEDIDOS: pestañas Activos e Historial (con botón "Volver a pedir").

===== FRONT-END: VISTA DEL REPARTIDOR =====

Barra inferior con: Inicio, Ruta, Disponibles, Mapa, Análisis, Perfil.

1. INICIO (Dashboard): tarjetas con indicadores del día (pedidos, completados, pendientes, tiempo, ingresos) y un resumen con la zona y hora pico.
2. PEDIDOS DISPONIBLES: pedidos "confirmados" sin tomar, cada uno con cliente, zona, productos y total, y un botón "Aceptar e iniciar entrega" (PATCH ../acceptar).
3. HOJA DE RUTA: lista numerada de entregas ordenadas por cercanía, agrupadas por zona (un color por zona), con dirección, hora y estado. Botón "Marcar como entregado" por parada (PATCH ../entregar), barra de progreso y botón "Optimizar".
4. MAPA DE CALOR: demanda por zonas con ranking en porcentajes (resuelve "¿qué zonas piden más?").
5. ANÁLISIS DE DEMANDA: gráfico de pedidos por día y de demanda por hora, con la franja pico destacada (resuelve "¿a qué hora hay más demanda?").
6. PERFIL: datos de Matías y métricas acumuladas.



```
===== ESQUEMA SQL (úsalo tal cual) =====

-- DDL: estructura

CREATE TABLE zona (
    id INTEGER PRIMARY KEY, nombre TEXT NOT NULL UNIQUE, color TEXT NOT NULL,
    centro_lat REAL NOT NULL, centro_lng REAL NOT NULL
);

CREATE TABLE cliente (
    id INTEGER PRIMARY KEY, nombre TEXT NOT NULL, direccion TEXT NOT NULL,
    zona_id INTEGER NOT NULL REFERENCES zona(id), lat REAL NOT NULL, lng REAL NOT NULL
);

CREATE TABLE producto (
    id INTEGER PRIMARY KEY, nombre TEXT NOT NULL, descripcion TEXT,
    precio INTEGER NOT NULL, categoria TEXT NOT NULL
);

CREATE TABLE repartidor (
    id INTEGER PRIMARY KEY, nombre TEXT NOT NULL,
    origen_lat REAL NOT NULL, origen_lng REAL NOT NULL
);

CREATE TABLE pedido (
    id INTEGER PRIMARY KEY,
    cliente_id INTEGER NOT NULL REFERENCES cliente(id),
    zona_id INTEGER NOT NULL REFERENCES zona(id),
    fecha TEXT NOT NULL, hora INTEGER NOT NULL,
    estado TEXT NOT NULL DEFAULT 'confirmado', -- confirmado | en_ruta | entregado
    total INTEGER NOT NULL DEFAULT 0, orden_ruta INTEGER
);

CREATE TABLE detalle_pedido (
    id INTEGER PRIMARY KEY,
    pedido_id INTEGER NOT NULL REFERENCES pedido(id),
    producto_id INTEGER NOT NULL REFERENCES producto(id),
    cantidad INTEGER NOT NULL, precio_unitario INTEGER NOT NULL
);

-- Vistas para las analíticas
```



```
CREATE VIEW v_pedidos_por_zona AS

    SELECT z.nombre, z.color, COUNT(p.id) AS total
    FROM pedido p JOIN zona z ON z.id = p.zona_id
    GROUP BY z.id ORDER BY total DESC;

CREATE VIEW v_demanda_por_hora AS

    SELECT hora, COUNT(*) AS total FROM pedido GROUP BY hora ORDER BY hora;

-- DML: datos de prueba

INSERT INTO zona (id,nombre,color,centro_lat,centro_lng) VALUES

    (1,'Centro','#2E5BFF',-33.445,-70.650),
    (2,'Providencia','#17C3B2',-33.427,-70.610),
    (3,'Las Condes','#3A86FF',-33.408,-70.570),
    (4,'Ñuñoa','#4895EF',-33.456,-70.600),
    (5,'Santiago Poniente','#5390D9',-33.450,-70.690),
    (6,'Vitacura','#48BFE3',-33.390,-70.560);

INSERT INTO repartidor (id,nombre,origen_lat,origen_lng) VALUES

    (1,'Matías Aravena',-33.4489,-70.6693);

INSERT INTO producto (id,nombre,descripcion,precio,categoria) VALUES

    (1,'Pastel de Choclo Casero','Tradicional pastel chileno horneado en greda',6800,'Platos Caseros'),

    (2,'Cazuela de Vacuno','Caldo casero de vacuno con choclo y zapallo',6200,'Platos Caseros'),

    (3,'Porotos con Riendas','Porotos guisados con tallarines y longaniza',5900,'Platos Caseros'),

    (4,'Charquicán de Verduras','Guiso de papas machacadas con huevo frito',5200,'Vegetarianos'),

    (5,'Humitas de la Casa','Choclo tierno molido en hojas (2 unidades)',4900,'Vegetarianos'),

    (6,'Ensalada César','Lechugas frescas con aderezo César y garbanzos',4600,'Vegetarianos'),

    (7,'Bebida 350ml','Bebida en lata fría',1500,'Bebidas');

INSERT INTO cliente (id,nombre,direccion,zona_id,lat,lng) VALUES

    (1,'Carla Soto','Av. Apoquindo 4500, Las Condes',3,-33.409,-70.571),
    (2,'Diego Rojas','Av. Providencia 1200, Providencia',2,-33.426,-70.611),
    (3,'Fernanda Lillo','Moneda 920, Centro',1,-33.444,-70.651),
    (4,'José Pérez','Irrarrázaval 3400, Ñuñoa',4,-33.455,-70.601),
```



```
(5,'María Díaz','Alameda 1100, Santiago Poniente',5,-33.451,-70.691),  
(6,'Tomás Vera','Av. Vitacura 5600, Vitacura',6,-33.391,-70.561);
```

- Genera además ~40 pedidos de ejemplo del día de hoy con sus detalles, repartidos en
- las 6 zonas y con las horas concentradas entre las 11:00 y las 14:00. Mezcla estados:
- algunos "confirmado" (para Pedidos Disponibles), algunos "en_ruta" y algunos
- "entregado", para que el dashboard, la ruta y las analíticas tengan datos.

===== ENTREGABLE =====

Organiza el proyecto en /backend, /frontend y /db, con un README de ejecución. El dashboard, la hoja de ruta y las analíticas deben calcularse a partir de la base de datos. Verifica que un pedido creado desde la vista del Cliente aparezca en "Pedidos Disponibles" de la vista del Repartidor.

-Prompts modificaciones del repartidor

Promots.8

Actúa como desarrollador full-stack. Crea la aplicación web completa "RutaSmart", para un negocio de reparto de almuerzos caseros en bicicleta en Santiago de Chile. El usuario principal es Matías (el repartidor) y también la usan sus clientes (oficinistas). La app debe recibir pedidos, agruparlos por zona, calcular una ruta de entrega optimizada y mostrar un panel de analíticas. Todo en español.

Parte 1 – Flujo del Cliente

1. Pantalla inicial – Inicio de sesión (Cliente):

- La primera pantalla que verá el cliente al abrir la aplicación será la de iniciar sesión.
- El cliente debe ingresar su correo y contraseña para autenticarse.
- Una vez autenticado, se redirige automáticamente a la pestaña Inicio.
- Mostrar mensaje: “Bienvenido, [nombre del cliente]” en la parte superior.
- En la pestaña Perfil, mostrar la foto del cliente junto a su nombre.
- Incluir botón “Cerrar sesión” que invalide la sesión y redirija a la pantalla de login.

2. Pedido del cliente:

- Desde la pestaña Inicio, el cliente selecciona productos y confirma su pedido.
- Al confirmar, el pedido queda registrado en la base de datos con estado inicial “Preparación”.



- El cliente presiona el botón Seguimiento (antes Pedidos), donde se muestra solo el pedido activo realizado tras iniciar sesión.
- En Seguimiento se despliega el estado cronológico del pedido y el mapa en tiempo real con la ruta del repartidor hacia la dirección del cliente.

3. Notificaciones al cliente (estilo PedidosYa):

- “Tu pedido ha sido recibido.”
- “Tu pedido está siendo preparado.”
- “Tu pedido fue entregado al repartidor.”
- “Matías aceptó tu pedido.”
- “Tu pedido está en camino.”
- “Tu repartidor llegará en aproximadamente 5 minutos.”
- “Tu pedido fue entregado correctamente.”

Parte 2 – Flujo del Usuario (Repartidor)

1. Pantalla inicial – Inicio de sesión (Usuario/Repartidor):

- Al pasar a la perspectiva del repartidor, la primera pantalla será la de iniciar sesión.
- El repartidor debe ingresar su correo y contraseña para autenticarse.
- Una vez autenticado, se redirige automáticamente a la pestaña Inicio.
- Mostrar mensaje: “Bienvenido, [nombre del repartidor]” en la parte superior.

2. Gestión del pedido (Repartidor):

- En la pestaña Inicio, el repartidor verá los pedidos disponibles.
- Debe aceptar el pedido del cliente (ejemplo: Julián Cortés).
- Al aceptar, el estado del pedido cambia a “Entregado al repartidor” y se refleja en tiempo real en la perspectiva del cliente.
- El repartidor continúa con el flujo: En camino → Entregado, con actualizaciones visibles en el cliente.

3. Panel del repartidor:

- Barra inferior con íconos: Inicio, Ruta, Mapa, Análisis, Perfil.
- En la parte superior mostrar métricas dinámicas:
 - Pedidos Hoy
 - Completados
 - Pendientes
 - Tiempo Est.

Funcionalidades comunes

- Mapa:



- Reemplazar mapa oscuro por mapa real de Santiago de Chile (Leaflet.js + OpenStreetMap).
 - Mostrar calles, comunas y referencias reconocibles (Santiago Centro, Providencia, Las Condes, Ñuñoa, etc.).
 - Funcionalidades: zoom, arrastrar, centrar vista.
 - Marcadores modernos con borde blanco y sombra suave.
- Fecha y estado del servicio:
 - En la esquina superior derecha mostrar la fecha actual en formato: "Viernes 19 de Junio".
 - Debajo: punto verde + texto "En servicio" en color verde.
 - A la derecha: ícono de calendario azul.
 - Fecha actualizada automáticamente según el sistema.
- Coincidencia imágenes-productos:
 - Cada producto debe mostrar su imagen real (ejemplo: Completo → foto de completo chileno).
 - No modificar los colores de las imágenes exportadas desde Stitch.
- Base de datos SQL:
 - Tablas:
 - clientes (con foto_url)
 - sesiones
 - pedidos
 - detalle_pedidos
 - repartidores
 - notificaciones
 - Ejemplo: cliente Julián Cortés con foto cargada en foto_url, pedido activo en estado "preparación", y detalle con productos.